

Universidad Nacional de Ingeniería

Facultad de Ciencias

Programación Concurrente y Distribuida

PC 4

Dog Messenger en Red Local

Sockets TCP, QR de vinculación, envío de archivos y clonación de dispositivo



Estudiantes: Cesar Omar Lopez Arteaga
Ivan Marcelo Urbano Nieves

Proyecto: Dog Messenger

Lenguajes: Kotlin, Java, XML, Gradle y Maven

Plataformas: Android y Desktop

Comunicación: Socket TCP sobre red local

Índice

1	Introduccion	2
1.1	Problema trabajado	2
1.2	Objetivo general	2
1.3	Objetivos especificos	2
2	Estructura general del proyecto	2
2.1	Archivos revisados	3
3	Arquitectura de comunicacion	3
3.1	Regla de red local	5
3.2	Seleccion de IP en desktop	5
4	Flujo de comunicacion TCP	5
4.1	Servidor TCP	7
4.2	Cliente TCP	7
5	Protocolo de mensajes, archivos y configuracion	8
5.1	Formato de mensaje	8
5.2	Formato de archivo	8
5.3	Formato de configuracion de tienda	9
6	QR de vinculacion y QR de clonacion	9
6.1	Diferencia entre vincular y clonar	9
6.2	Modo silencioso	10
7	Casos de uso	10
7.1	Caso de uso de clonacion	11
7.2	Caso de uso de archivos	12
7.3	Caso de uso de Tienda Virtual	12
8	Diagramas de clase	13
8.1	Clases Android	13
8.2	Clases desktop	15
9	Implementacion por componente	17
9.1	Servidor Android	17
9.2	Cliente Android	17
9.3	Servidor desktop	17
9.4	Cliente desktop	17
9.5	Server50Tienda	17
9.6	VentasIA	17
9.7	Cliente de Tienda Virtual	18
10	Validacion y pruebas	18
10.1	Pruebas revisadas sobre el proyecto	18
10.2	Comandos de ejecucion desktop	19
10.3	Orden de levantamiento recomendado	19
10.4	Regla de firewall	20
11	Conclusiones	20
	Anexo A: Resumen de ejecucion recomendado	20
	Anexo B: Puertos usados	21

1 Introduccion

El laboratorio se centra en una aplicacion distribuida de mensajería sobre red local. La idea general es que un dispositivo funcione como servidor y otros dispositivos se conecten como clientes. Una vez establecida la conexión, cualquier mensaje enviado por un cliente se reenvía a todos los clientes conectados, logrando un comportamiento de grupo de chat.

A diferencia de una aplicacion que depende de Internet o de un servidor externo, Dog Messenger trabaja en una red LAN. Esto significa que la PC, el celular o los emuladores no deben usar direcciones aisladas entre si, sino una IP privada visible dentro del mismo segmento de red. Por ejemplo, si el servidor esta en 192.168.0.164, el movil debe estar en una direccion similar como 192.168.0.x.

El proyecto enviado contiene cuatro componentes principales: servidor Android, cliente Android, servidor desktop y cliente desktop. Todos comparten la misma idea de comunicacion, pero cada plataforma adapta la interfaz, la lectura de QR y el almacenamiento de archivos recibidos.

1.1 Problema trabajado

El problema principal es implementar una comunicacion distribuida simple, pero funcional, entre diferentes dispositivos. Tambien se busca resolver el problema de vinculacion: ingresar manualmente IP y puerto puede generar errores. Por eso se incorporan codigos QR que contienen la direccion de conexión en el formato `dogmsg://IP:PUERTO`.

Luego se agrega una funcion mas especifica: clonar dispositivo. En ese caso, el cliente que ya esta conectado genera un QR especial. El nuevo dispositivo escanea ese QR, copia la misma IP y puerto, y se conecta al mismo grupo. La diferencia es que la conexión se realiza en modo silencioso para que parezca una clonacion y no una entrada normal de otro usuario.

1.2 Objetivo general

Implementar y documentar el funcionamiento de Dog Messenger como sistema de mensajería en red local, usando sockets TCP, interfaces Android/Desktop, codigos QR y clonacion de dispositivo.

1.3 Objetivos especificos

- Revisar la estructura del código enviado y separar las responsabilidades de servidor, cliente, protocolo, QR e interfaz grafica.
- Explicar la comunicacion TCP entre servidor y clientes usando puerto configurable, principalmente el puerto 8189.
- Documentar el protocolo de mensajes MSG y de archivos FILE.
- Generar diagramas de comunicacion, casos de uso y diagramas de clase a partir del código real.
- Explicar la vinculacion por QR y la clonacion con `clone=1`.
- Resaltar la validacion de red local para evitar conexiones por IP incorrecta o por adaptadores virtuales.

2 Estructura general del proyecto

El archivo del proyecto contiene dos aplicaciones Android y dos aplicaciones desktop. Las aplicaciones Android estan desarrolladas con Kotlin y Gradle. Las aplicaciones desktop estan desarrolladas en Java Swing;

el cliente desktop utiliza Maven para incorporar lectura de QR con camara mediante `webcam-capture` y `ZXing`.

Cuadro 1: Componentes principales del proyecto Dog Messenger

Componente	Tecnologia	Funcion principal
Server50	Android / Kotlin	Aplicacion servidor para Android. Abre un <code>ServerSocket</code> , acepta clientes y reenvia mensajes.
Cliente50	Android / Kotlin	Aplicacion cliente para Android. Se conecta al servidor, envia mensajes, adjunta archivos, lee QR y valida red local.
Servidor50DesktopGUI	Java Swing	Servidor desktop con pantalla de presentacion, seleccion automatica de IP LAN, envio de mensajes y archivos.
DesktopClient50GUI	Java Swing + Maven	Cliente desktop con pantalla de presentacion, conexion TCP, lectura de QR con camara o ingreso manual.

2.1 Archivos revisados

Los archivos mas importantes para el funcionamiento del laboratorio son los siguientes:

Cuadro 2: Archivos principales revisados del codigo

Archivo	Responsabilidad
Server50/.../MainActivity.kt	Interfaz del servidor Android, inicio/parada del servidor, envio de mensajes y archivos.
Server50/.../TCPServer50.kt	Servidor TCP Android; acepta conexiones y maneja hasta 10 clientes activos.
Server50/.../TCPServerThread50.kt	Hilo independiente por cliente conectado al servidor Android.
Cliente50/.../MainActivity.kt	Interfaz del cliente Android, conexion, envio, recepcion, QR y validacion LAN.
Cliente50/.../TCPClient50.kt	Cliente TCP Android; abre socket, envia lineas y escucha mensajes del servidor.
Cliente50/.../QrConnectionParser.kt	Parser del contenido QR <code>dogmsg://IP:PUERTO</code> y de clonacion.
Cliente50/.../ScanQrActivity.kt	Lectura de QR desde la camara Android y envio automatico de IP/puerto al <code>MainActivity</code> .
ChatProtocol.kt	Codificacion, decodificacion y visualizacion de mensajes y archivos.
DesktopServer50GUI.java	Servidor desktop con Swing, splash de 4 segundos, seleccion de IP LAN y broadcast.
DesktopClient50GUI.java	Cliente desktop con Swing, camara para QR, ingreso manual de QR y modo silencioso de clonacion.

3 Arquitectura de comunicacion

La arquitectura funciona bajo el modelo cliente-servidor. El servidor puede ser Android o desktop. Los clientes pueden ser Android o desktop. El punto importante es que todos deben estar conectados a la misma red local. No se debe usar una IP de WSL, VirtualBox, Docker, Hyper-V o una red NAT que no pueda ser vista por el movil.

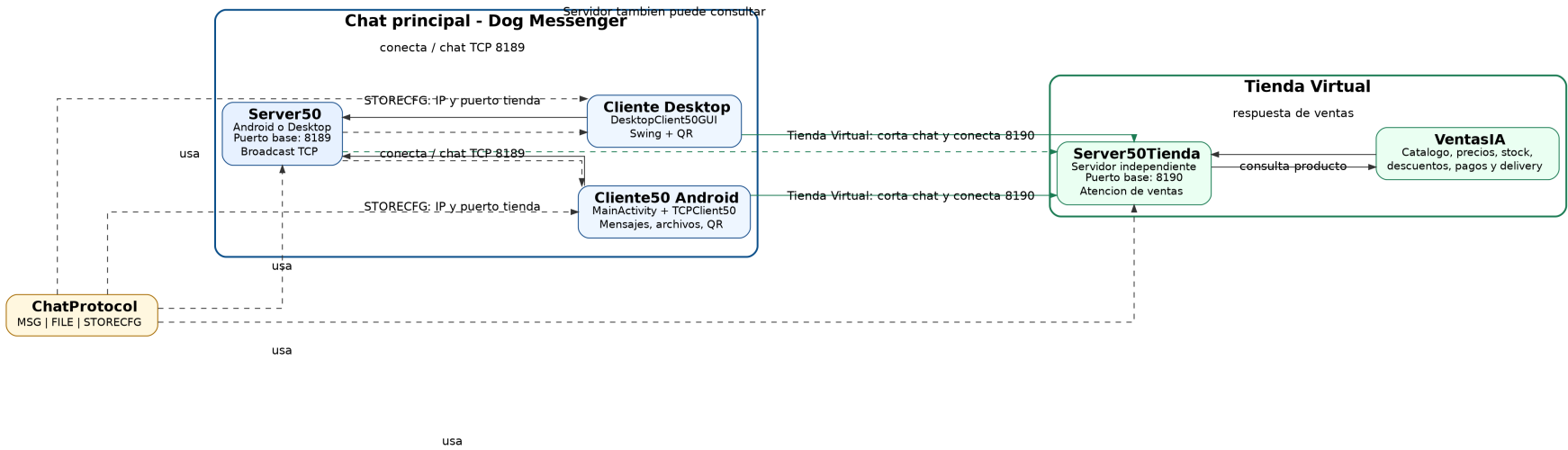


Figura 1: Arquitectura general de Dog Messenger sobre red local.

3.1 Regla de red local

La regla usada en esta version es directa: servidor y cliente deben estar en la misma red local. El codigo valida que la IP del servidor sea IPv4 privada y que coincida el prefijo /24. Es decir, se comparan los tres primeros octetos. Por ejemplo:

Cuadro 3: Ejemplos de conexion valida e invalida

Servidor	Cliente	Resultado	Motivo
192.168.0.164	192.168.0.163	Valido	Ambos pertenecen al segmento 192.168.0.x.
192.168.1.20	192.168.1.35	Valido	Ambos pertenecen al segmento 192.168.1.x.
172.20.45.184	10.105.164.76	Invalido	Son redes privadas, pero no pertenecen al mismo segmento local.
192.168.56.1	192.168.0.163	Invalido	192.168.56.1 suele ser VirtualBox Host-Only, no la red Wi-Fi real.

3.2 Seleccion de IP en desktop

En las aplicaciones desktop se corrige el problema de tomar una IP incorrecta. El codigo prioriza direcciones 192.168.x.x y penaliza adaptadores virtuales como VirtualBox, VMware, Hyper-V, WSL, Docker, Bluetooth o interfaces tipo tunnel. Ademàs, en el servidor desktop el campo IP queda editable para poder escribir manualmente la IP Wi-Fi real si fuera necesario.

4 Flujo de comunicacion TCP

La comunicacion se hace mediante sockets TCP. El servidor abre un puerto, por defecto 8189, y queda esperando conexiones. Cuando un cliente se conecta, el servidor crea un hilo para atenderlo. Cada mensaje recibido por un cliente se reenvia a todos los clientes activos. Este comportamiento corresponde al broadcast del chat.

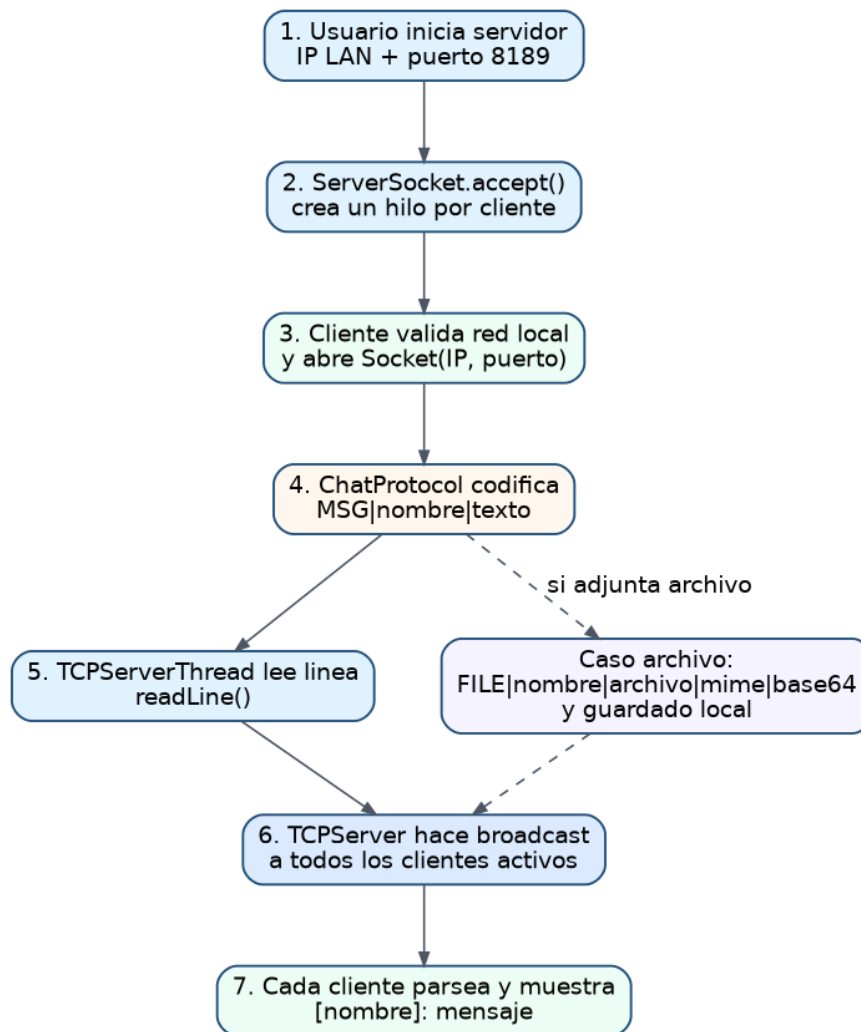


Figura 2: Flujo de comunicacion TCP para mensajes y archivos.

4.1 Servidor TCP

En Android, la clase `TCPServer50` crea un `ServerSocket` con el puerto configurado. Despues ejecuta un ciclo de aceptacion con `accept()`. Por cada cliente aceptado se obtiene un identificador disponible y se crea un `TCPServerThread50`. Esto permite que cada cliente tenga su propio hilo de lectura.

```
1 serverSocket = ServerSocket(portIp__)
2 while (running) {
3     val client: Socket = serverSocket!!.accept()
4     val idDisponible = obtenerIdDisponible()
5     val hiloCliente = TCPServerThread50(client, this, idDisponible, ipCliente)
6     registrarCliente(idDisponible, hiloCliente)
7     Thread(hiloCliente).start()
8 }
```

Listing 1: Idea central del servidor TCP Android.

La cantidad maxima configurada es de 10 clientes. Cuando llega un mensaje, el servidor llama a `broadcastRaw(message)` y lo envia a cada cliente registrado.

4.2 Cliente TCP

En Android, la clase `TCPClient50` abre un `Socket` hacia la IP y puerto del servidor. La conexion tiene un timeout de 5000 ms. Luego crea un `PrintWriter` para enviar lineas y un `BufferedReader` para escuchar los mensajes del servidor.

```
1 val nuevoSocket = Socket()
2 nuevoSocket.connect(InetSocketAddress(host, serverPort), CONNECT_TIMEOUT_MS)
3 out = PrintWriter(BufferedWriter(OutputStreamWriter(nuevoSocket.getOutputStream())), true)
4 input = BufferedReader(InputStreamReader(nuevoSocket.getInputStream()))
```

Listing 2: Conexion del cliente TCP Android.

En desktop ocurre la misma idea con Java: el cliente usa `Socket`, el servidor usa `ServerSocket`, y la interfaz grafica se mantiene separada del hilo de red para evitar bloquear la ventana.

5 Protocolo de mensajes, archivos y configuracion

El protocolo se mantiene simple porque la comunicacion se realiza por lineas de texto. Cada mensaje se codifica con campos separados por el simbolo |. En esta nueva revision se agregan tres tipos principales de paquetes:

Cuadro 4: Tipos de paquetes usados por el protocolo actualizado

Tipo	Formato	Uso
MSG	MSG sender text	Envio de mensajes normales de chat o consultas de tienda.
FILE	FILE sender fileName mimeType base64	Envio de archivos adjuntos codificados en Base64.
STORECFG	STORECFG ipTienda puertoTienda	Configuracion interna para indicar a los clientes donde esta Server50Tienda .

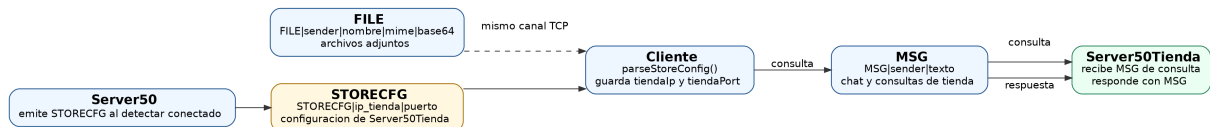


Figura 3: Relacion entre el protocolo de chat y la configuracion de Tienda Virtual.

5.1 Formato de mensaje

Para un mensaje normal se usa el tipo MSG. El nombre del emisor y el texto se codifican con URL encoding para evitar problemas con espacios, tildes o caracteres especiales.

```
1 MSG|nombreCodificado|textoCodificado
```

Listing 3: Formato logico del mensaje normal.

Por ejemplo, si el usuario Cesar envia Hola, el mensaje logico es:

```
1 MSG|Cesar|Hola
```

Listing 4: Ejemplo de mensaje de texto.

5.2 Formato de archivo

Para archivos se usa el tipo FILE. El archivo se convierte a Base64 y se envia en la misma linea. Esto permite transmitir imagenes, documentos u otros archivos sin abrir otro canal TCP.

```
1 FILE|nombreCodificado|archivoCodificado|mimeTypeCodificado|contenidoBase64
```

Listing 5: Formato logico del envio de archivo.

Cuando el receptor detecta un paquete de tipo FILE, reconstruye los bytes y los guarda. En Android se intenta guardar primero en Descargas publicas, dentro de **DogMessengerRecibidos**. Si no se puede, se guarda en una carpeta privada de la aplicacion.

5.3 Formato de configuracion de tienda

La novedad de esta revision es el paquete `STORECFG`. Este paquete no debe mostrarse como mensaje visible del chat. Su funcion es guardar internamente la IP y puerto de `Server50Tienda`. Cuando el usuario presione `Tienda Virtual`, la aplicacion ya sabe a que servidor debe conectarse.

```
1 STORECFG|ipTiendaCodificada|puertoTienda
```

Listing 6: Formato logico de configuracion de tienda.

Por ejemplo, si `Server50Tienda` se encuentra en la misma PC con IP `192.168.0.164` y puerto `8190`, el paquete enviado por `Server50` puede representarse como:

```
1 STORECFG|192.168.0.164|8190
```

Listing 7: Ejemplo de configuracion de tienda.

En el cliente Android, `ChatProtocol.parseStoreConfig(raw)` retorna un par `Pair<String, Int>` con la IP y el puerto. Luego `MainActivity` guarda estos datos en `tiendaIp` y `tiendaPort`. En el cliente desktop, la misma idea se implementa con una estructura de configuracion interna para abrir `TiendaClientWindow`.

6 QR de vinculacion y QR de clonacion

Los codigos QR simplifican la conexion. En vez de escribir IP y puerto, el usuario puede escanear un QR que contiene los datos de conexion. El formato usado es:

```
1 dogmsg://192.168.0.164:8189
```

Listing 8: QR normal para unirse al grupo.

Para clonar dispositivo se agrega el parametro `clone=1`:

```
1 dogmsg://192.168.0.164:8189?clone=1
```

Listing 9: QR para clonar dispositivo.

6.1 Diferencia entre vincular y clonar

Cuadro 5: Comparacion entre vinculacion normal y clonacion

Operacion	QR aceptado	Comportamiento
Vincular dispositivo	<code>dogmsg://IP:PUERTO</code> o QR de clonacion	Carga IP y puerto, se conecta y puede anunciar <code>[nombre] conectado</code> .
Scanear p/clonar dispositivo	Solo QR con <code>clone=1</code>	Se conecta en modo silencioso y no anuncia nuevamente la conexion.
QR p/clonar dispositivo	Se genera desde un cliente conectado	Copia la IP y puerto del servidor actual, agregando <code>?clone=1</code> .

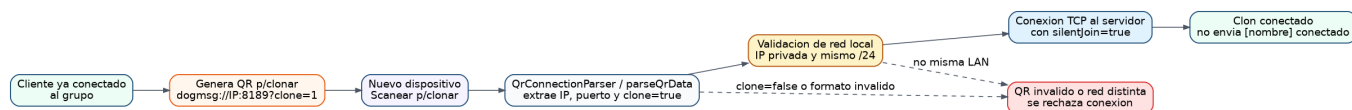


Figura 4: Flujo de clonacion mediante QR especial.

6.2 Modo silencioso

En el cliente Android, el QR se procesa y se pasan cuatro datos al **MainActivity**: IP, puerto, auto conexion y **silentJoin**. Si **silentJoin** es verdadero, el cliente se conecta pero no envia el mensaje de estado **conectado**. Esta parte permite que la entrada sea tratada como una clonacion.

```

1  silentSession = silentJoinOnNextConnect
2  if (!silentSession) {
3      mTcpClient?.sendMessage(ChatProtocol.encodeMessage(nombreActual(), "conectado"))
4  }

```

Listing 10: Efecto del modo silencioso al conectarse.

7 Casos de uso

Los casos de uso principales ahora se dividen en tres grupos: chat principal, clonacion/vinculacion y Tienda Virtual. La Tienda Virtual aparece como una extension del flujo principal, pero usa otro servidor TCP.

Cuadro 6: Casos de uso principales del sistema actualizado

Codigo	Caso de uso	Descripcion
UC-01	Iniciar servidor de chat	El usuario servidor inicia Server50 en el puerto configurado, normalmente 8189 .
UC-02	Conectar cliente	El cliente ingresa IP/puerto o escanea QR para conectarse al chat.
UC-03	Enviar mensaje	Un cliente envia un paquete MSG ; el servidor lo reenvia a los demas.
UC-04	Enviar archivo	Un cliente envia un paquete FILE ; el receptor lo reconstruye y lo guarda.
UC-05	Generar QR de grupo	El dispositivo conectado genera un QR con IP y puerto del servidor.
UC-06	Vincular dispositivo	Otro dispositivo escanea el QR normal y se conecta como usuario visible.
UC-07	Clonar dispositivo	Otro dispositivo escanea un QR con clone=1 y se conecta en modo silencioso.
UC-08	Iniciar Server50Tienda	Se levanta el servidor independiente de tienda, normalmente por el puerto 8190 .
UC-09	Enviar configuracion de tienda	Server50 envia STORECFG con la IP y puerto de Server50Tienda .
UC-10	Entrar a Tienda Virtual	El cliente corta la comunicacion con Server50 y abre una conexion con Server50Tienda .
UC-11	Consultar productos	El cliente pregunta por catalogo, precios, stock, descuentos, pagos o envios.
UC-12	Responder con VentasIA	Server50Tienda procesa la consulta con VentasIA y responde al cliente.
UC-13	Volver al chat anterior	El cliente cierra la tienda y reconecta con el chat principal si antes estaba conectado.
UC-14	Cerrar aplicacion	El usuario cierra la interfaz activa y se detienen los sockets correspondientes.

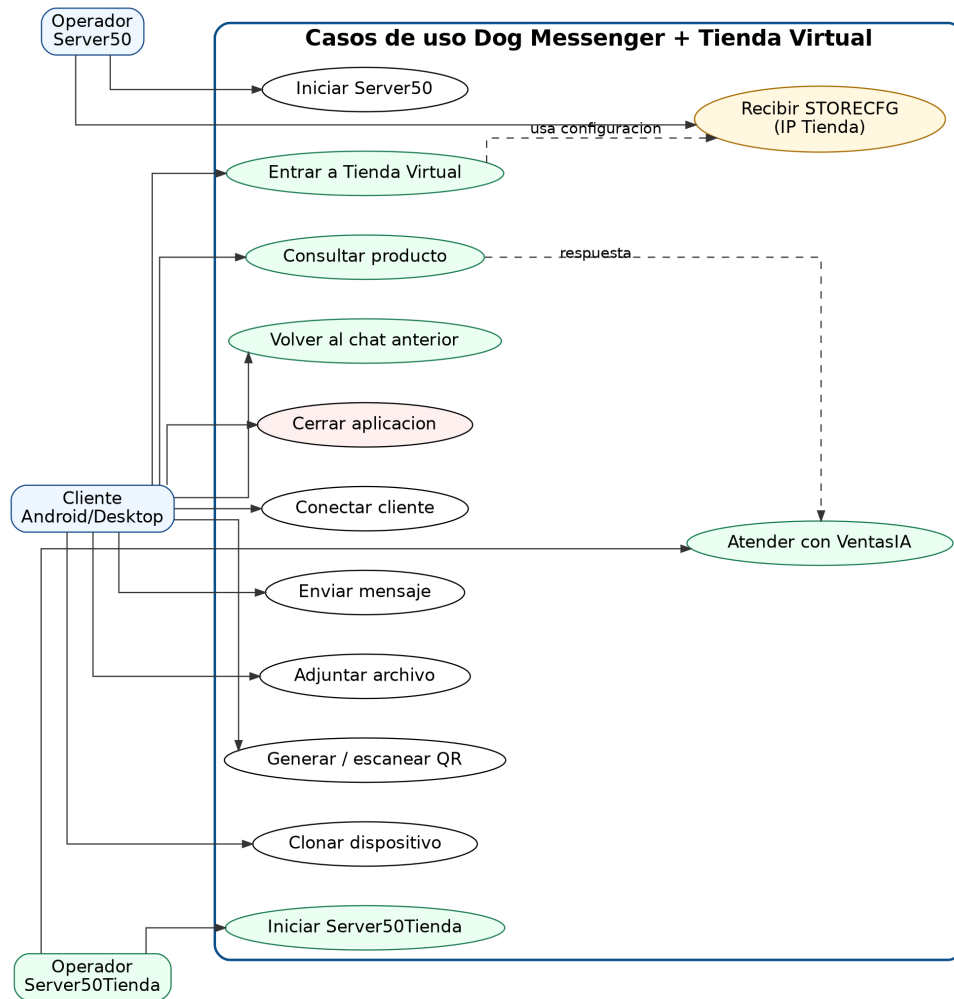


Figura 5: Casos de uso generales de Dog Messenger con Tienda Virtual.

7.1 Caso de uso de clonacion

El caso de clonacion se activa cuando un cliente conectado selecciona la opcion de generar QR para clonar dispositivo. El QR contiene los datos del servidor y el parametro `clone=1`. El dispositivo que escanea ese QR se conecta al mismo servidor sin anunciar una entrada normal al grupo.

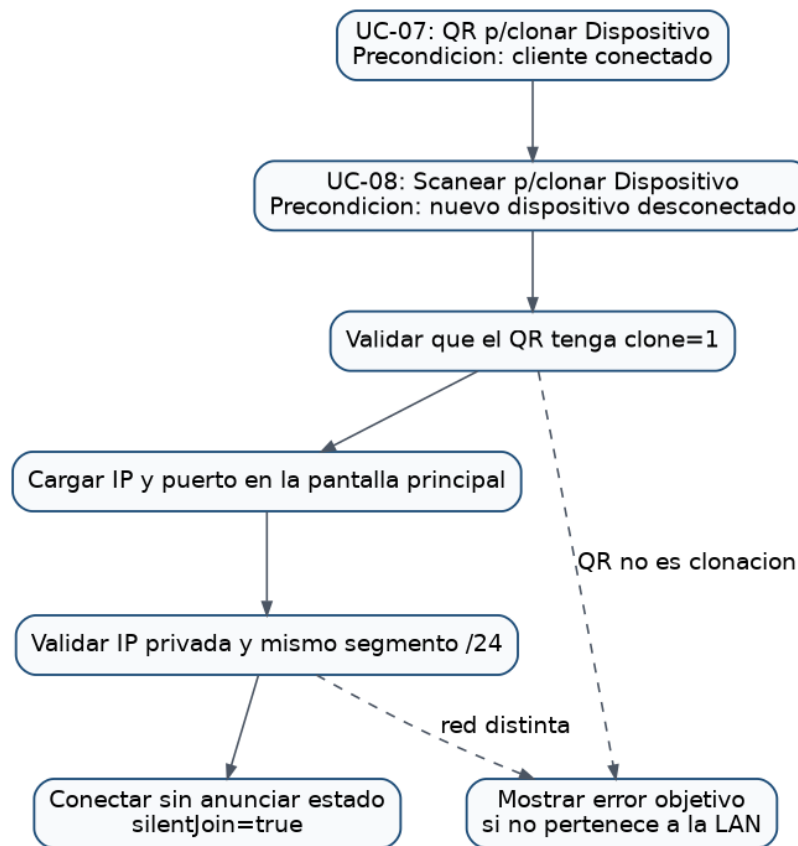


Figura 6: Caso de uso para clonacion de dispositivo mediante QR.

7.2 Caso de uso de archivos

El envio de archivos usa el mismo socket TCP. La aplicacion selecciona un archivo, lee sus bytes, lo codifica en Base64 y lo envia dentro de un paquete FILE. En el receptor se decodifica y se guarda en una carpeta disponible.

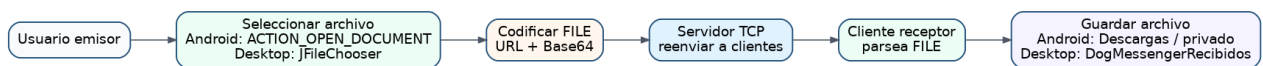


Figura 7: Caso de uso para envio y recepcion de archivos.

7.3 Caso de uso de Tienda Virtual

La Tienda Virtual no trabaja sobre el socket del chat. Primero recibe la configuracion por STORECFG; luego, al presionar el boton **Tienda Virtual**, se abre una nueva conexcion con **Server50Tienda**. Al volver, la conexcion con tienda se cierra y se regresa al servidor anterior.

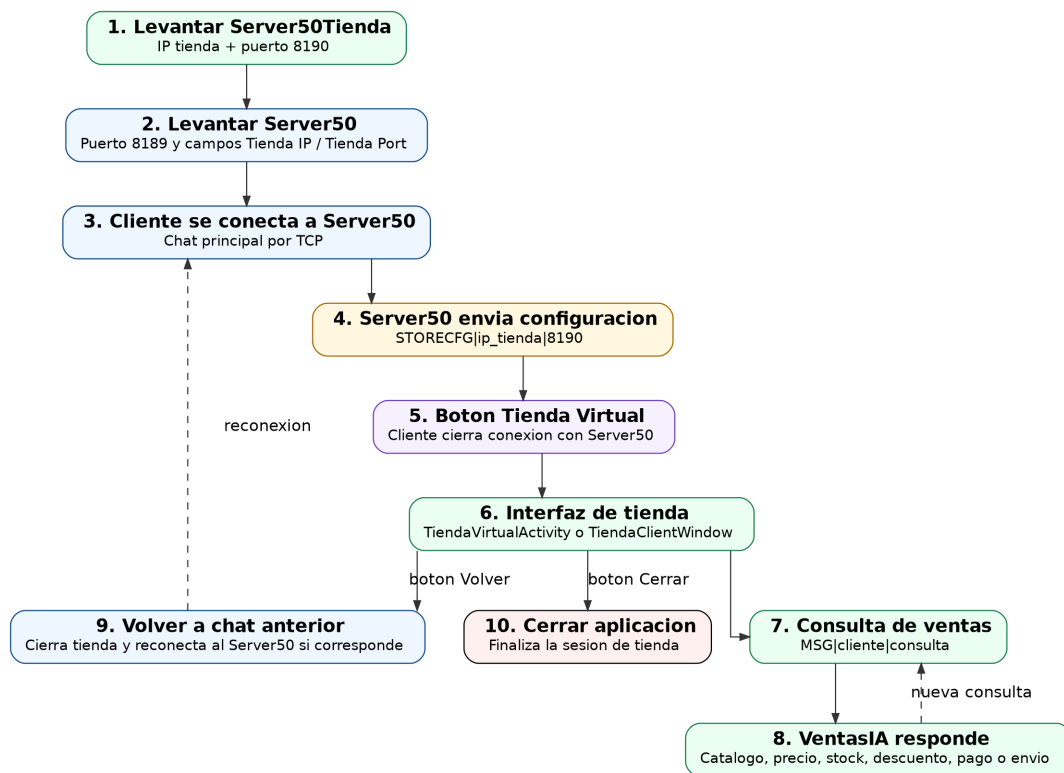


Figura 8: Caso de uso de cambio entre chat y Tienda Virtual.

8 Diagramas de clase

8.1 Clases Android

En Android se separa la interfaz grafica de la logica de red. `MainActivity` controla botones, mensajes, estados de conexion, envio de archivos y ahora tambien el boton `Tienda Virtual`. En el lado cliente se agregan las variables `tiendaIp` y `tiendaPort`, las cuales se llenan cuando llega un paquete `STORECFG`.

La clase `TiendaVirtualActivity` abre una interfaz distinta para consultar a `Server50Tienda`. Esta pantalla recibe por `Intent` la IP de la tienda, el puerto de la tienda, la IP anterior del chat y el puerto anterior. Asi puede cortar la comunicacion con la tienda y volver al chat principal. En el lado servidor, `TCPServer50` acepta clientes y `TCPServerThread50` atiende cada conexion en un hilo independiente.

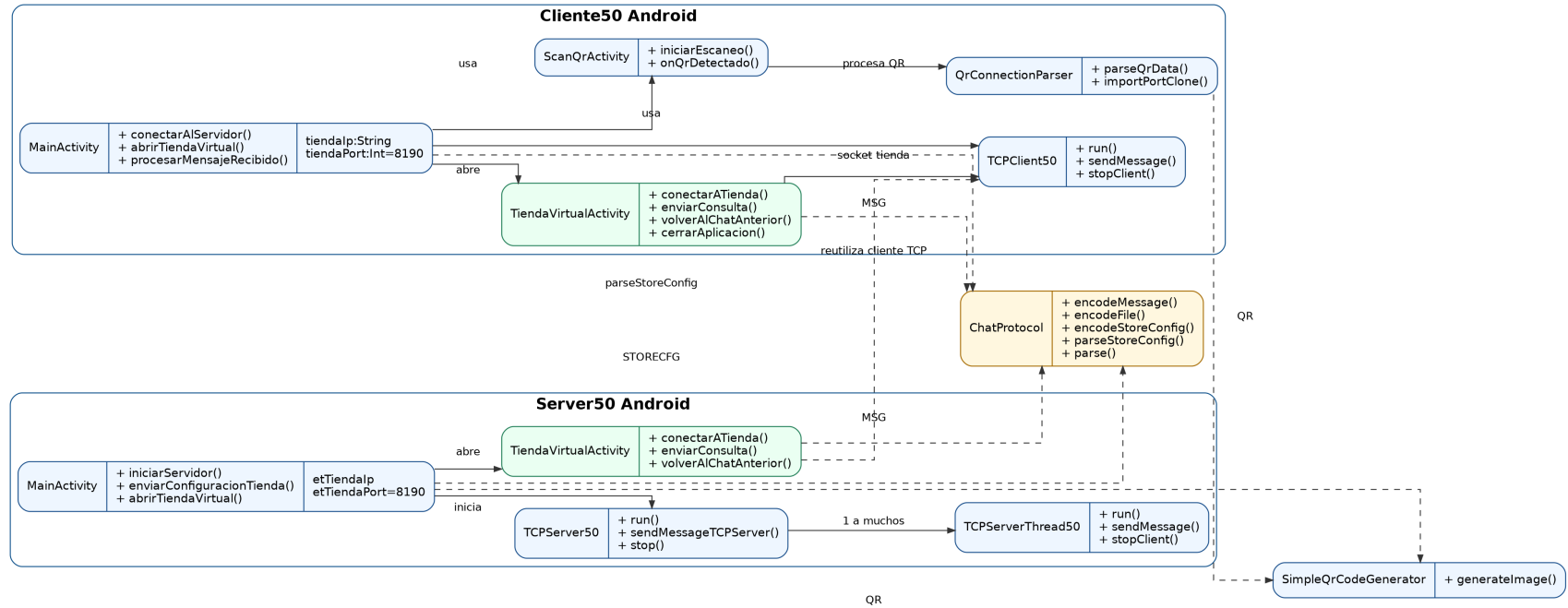


Figura 9: Diagrama de clases principales en las aplicaciones Android con Tienda Virtual.

8.2 Clases desktop

En desktop, la interfaz esta implementada con Java Swing. Se agrego una pantalla de presentacion de 4 segundos tanto para servidor como para cliente. El cliente desktop utiliza camara interna para QR, pero si no detecta camara, permite pegar manualmente el contenido del QR.

La nueva version agrega **TiendaClientWindow**, que funciona como ventana de consulta hacia **Server50Tienda**. Tambien se agrega **Server50TiendaGUI**, que levanta el servidor de tienda, acepta clientes con **StoreServer** y atiende cada conexion con **ClientHandler**. La respuesta comercial se genera mediante **VentasIA**, que contiene el catalogo y las reglas de respuesta.

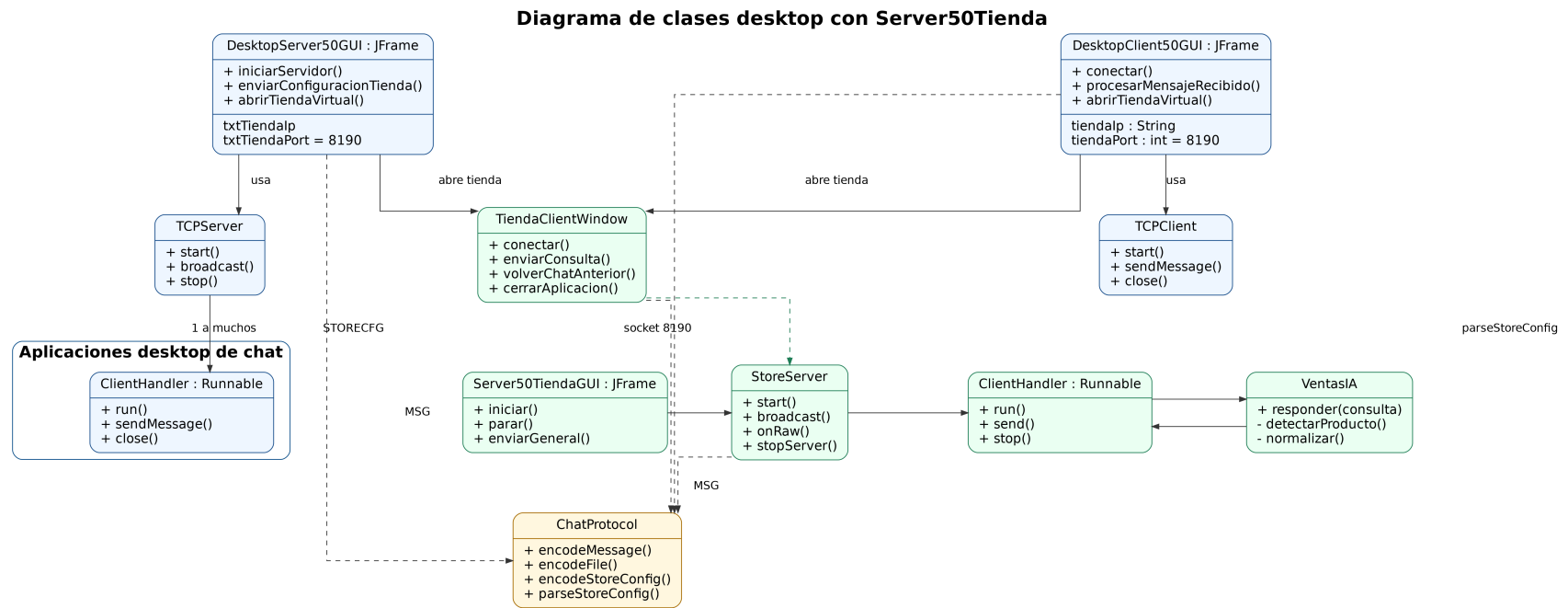


Figura 10: Diagrama de clases principales en las aplicaciones desktop con Server50Tienda.

9 Implementacion por componente

9.1 Servidor Android

El servidor Android inicia con `SplashActivity` y luego muestra la pantalla principal. Desde ahí se configura el puerto, se inicia el servidor y se habilita el envío de mensajes o archivos. El servidor utiliza `TCPServer50` para abrir el socket y `TCPServerThread50` para atender a cada cliente.

En la revisión actual se agrega la configuración de tienda. El servidor tiene campos para la IP y el puerto de `Server50Tienda`. Cuando un cliente envía el mensaje `conectado`, el servidor puede responder internamente con `STORECFG`. De esta manera, el cliente no necesita escribir manualmente la IP de la tienda.

9.2 Cliente Android

El cliente Android permite escribir nombre, IP y puerto. Antes de conectarse valida que la IP del servidor sea privada y que este en la misma red local. También puede recibir datos desde el escaneo QR. Cuando el QR contiene `clone=1`, se activa `silentJoinOnNextConnect`.

La nueva versión agrega el botón `Tienda Virtual`. Cuando el usuario lo presiona, se ejecuta `abrirTiendaVirtual()`. Este método guarda los datos del chat anterior, desconecta el socket actual y abre `TiendaVirtualActivity`. La pantalla de tienda se conecta a `Server50Tienda` usando `tiendaIp` y `tiendaPort`. Si el usuario presiona `Volver a chat anterior`, se cierra la conexión de tienda y se vuelve a `MainActivity` con los datos previos.

9.3 Servidor desktop

El servidor desktop usa Java Swing. Tiene pantalla de presentación, campo de IP editable, puerto configurable y botones para iniciar/parar servidor, enviar mensajes y adjuntar archivos. La función `obtenerIpLocal()` selecciona una IP LAN real y evita adaptadores virtuales.

En la nueva versión se agregan campos para `Tienda IP` y `Tienda Port`. Cuando se inicia el chat, estos datos permiten que el servidor envíe `STORECFG` a los clientes. También se agrega un botón `Tienda Virtual`, que permite abrir la ventana de tienda desde el lado desktop.

9.4 Cliente desktop

El cliente desktop mantiene la lógica de conexión TCP y agrega acceso a tienda. Puede recibir la configuración `STORECFG`, guardar la IP y puerto de `Server50Tienda`, y abrir `TiendaClientWindow`. Antes de entrar a tienda, el cliente cierra la comunicación con el chat para evitar que una misma interfaz mantenga dos conversaciones distintas al mismo tiempo.

9.5 Server50Tienda

`Server50Tienda` es un servidor independiente. Su objetivo no es reenviar mensajes de grupo, sino atender consultas de ventas. Internamente abre un `ServerSocket`, acepta clientes y crea un hilo `ClientHandler` para cada conexión. Cada consulta se interpreta como un mensaje de texto y luego se envía a `VentasIA`.

El puerto por defecto de la tienda es 8190. Esto evita conflicto con el chat principal, que usa normalmente 8189. Por esta separación, el usuario puede tener levantados los dos servidores dentro de la misma red local: uno para chat y otro para tienda.

9.6 VentasIA

`VentasIA` implementa una lógica simple de atención comercial. No es un modelo neuronal pesado; funciona como un motor de reglas sobre un catálogo definido en el código. La IA puede responder consultas sobre

productos, catalogo, precios, stock, descuentos, medios de pago, envio y garantia.

El catalogo revisado contiene productos como laptop, mouse, teclado, monitor, audifonos e impresora. Cuando el usuario pregunta por precio, stock o disponibilidad, **VentasIA** busca coincidencias por nombre de producto y devuelve una respuesta adecuada. Si la consulta es general, responde con una guia para que el cliente pregunte por catalogo, precios o formas de pago.

9.7 Cliente de Tienda Virtual

En Android, el cliente de tienda es **TiendaVirtualActivity**. En desktop, el cliente de tienda es **TiendaClientWindow**. Ambos cumplen el mismo proposito: mostrar una interfaz distinta al chat, conectarse a **Server50Tienda**, enviar consultas y recibir respuestas.

La interfaz tiene dos acciones importantes: **Volver a chat anterior** y **Cerrar aplicacion**. La primera cierra la conexion con tienda y regresa al chat principal. La segunda cierra la aplicacion completa. Este comportamiento cumple el requisito de cortar la comunicacion con un servidor antes de abrir o restaurar la comunicacion con otro.

10 Validacion y pruebas

10.1 Pruebas revisadas sobre el proyecto

Las pruebas se enfocan en verificar que los componentes se comuniquen correctamente dentro de la red local. Tambien se revisa que el nuevo servidor de tienda funcione separado del chat principal.

Cuadro 7: Pruebas funcionales del sistema actualizado

ID	Prueba	Resultado esperado
P-01	Iniciar Server50	El servidor abre el puerto 8189 y queda esperando clientes.
P-02	Conectar cliente Android	El cliente valida IP/puerto y establece socket TCP.
P-03	Conectar cliente desktop	El cliente Swing se conecta usando IP y puerto del servidor.
P-04	Enviar mensaje	El servidor recibe el paquete MSG y lo reenvia a los clientes conectados.
P-05	Enviar archivo	El receptor recibe FILE, decodifica Base64 y guarda el archivo.
P-06	Generar QR normal	Se crea un QR con <code>dogmsg://IP:PUERTO</code> .
P-07	Escanear QR normal	Se carga IP/puerto y se conecta como usuario visible.
P-08	Generar QR de clonacion	El QR incluye <code>clone=1</code> y nombre del usuario original.
P-09	Escanear QR de clonacion	El nuevo dispositivo adopta el nombre y entra en modo silencioso.
P-10	Validar red local	Se rechazan IP privadas que no pertenecen al mismo segmento /24.
P-11	Iniciar Server50Tienda	El servidor de tienda abre el puerto 8190 y queda esperando consultas.
P-12	Enviar STORECFG	Server50 envia a los clientes la IP y puerto de Server50Tienda .

ID	Prueba	Resultado esperado
P-13	Entrar a tienda	El cliente cierra el socket del chat y abre uno nuevo hacia Server50Tienda .
P-14	Consultar productos	VentasIA responde consultas sobre catalogo, precio, stock, descuentos y pago.
P-15	Volver al chat anterior	Se cierra la comunicacion con tienda y se reconecta al chat principal.
P-16	Firewall	Los puertos 8189 y 8190 quedan permitidos en la red local.

10.2 Comandos de ejecucion desktop

Para el servidor desktop principal:

```
1 cd Servidor50DesktopGUI
2 .\compile_and_run.ps1
```

Listing 11: Compilar y ejecutar servidor desktop principal en Windows.

Para el cliente desktop:

```
1 cd Cliente50\DesktopClient50GUI
2 mvn clean compile exec:java
```

Listing 12: Ejecutar cliente desktop con Maven.

Para el nuevo servidor de tienda:

```
1 cd Server50TiendaDesktopGUI
2 javac -encoding UTF-8 -d build\classes src\redes0k\*.java
3 java -cp build\classes redes0k.Server50TiendaGUI
```

Listing 13: Compilar y ejecutar Server50Tienda desktop.

10.3 Orden de levantamiento recomendado

El orden correcto de ejecucion para probar la Tienda Virtual es el siguiente:

1. Levantar primero **Server50Tienda** y anotar su IP y puerto, por ejemplo 192.168.0.164:8190.
2. Levantar luego **Server50** en el puerto de chat, por ejemplo 8189.
3. En **Server50**, colocar la IP y puerto de **Server50Tienda** en los campos de tienda.
4. Conectar los clientes al chat principal.
5. Verificar que los clientes reciban internamente **STORECFG**.
6. Presionar **Tienda Virtual** desde un cliente.
7. Consultar catalogo, precios o stock.
8. Presionar **Volver a chat anterior** para regresar al servidor **Server50**.

10.4 Regla de firewall

En Windows, si el servidor desktop no acepta conexiones, se debe permitir el trafico TCP en los puertos usados. Antes bastaba con permitir 8189; ahora tambien se debe permitir 8190 para **Server50Tienda**.

```
1 netsh advfirewall firewall add rule name="DogMessenger Chat 8189" dir=in action=allow  
   protocol=TCP localport=8189  
2 netsh advfirewall firewall add rule name="DogMessenger Tienda 8190" dir=in action=allow  
   protocol=TCP localport=8190
```

Listing 14: Reglas de firewall para chat y tienda virtual.

11 Conclusiones

1. Se implemento y documento un sistema distribuido de mensajeria en red local, donde un servidor TCP acepta varios clientes y reenvia mensajes mediante broadcast.
2. La arquitectura usa concurrencia porque cada cliente conectado es atendido por un hilo independiente. Esto permite que varios clientes puedan enviar y recibir mensajes sin bloquear la interfaz principal.
3. La validacion de red local es importante porque evita usar IP de adaptadores virtuales o redes que no son visibles desde el celular. Esta validacion reduce errores frecuentes de conexion.
4. El uso de QR simplifica la vinculacion, ya que evita escribir manualmente IP y puerto. El modo de clonacion extiende esta idea permitiendo copiar una sesion hacia otro dispositivo.
5. La nueva version agrega **Server50Tienda** como segundo servidor TCP independiente. Esto demuestra que el sistema puede extenderse a mas servicios dentro de la misma red local.
6. El protocolo se amplio con **STORECFG**. Este paquete permite que **Server50** informe a los clientes la IP y puerto de la tienda sin mostrar ese dato como mensaje normal del chat.
7. La Tienda Virtual cumple el requisito de cortar la comunicacion con **Server50**, abrir otra interfaz, conectarse a **Server50Tienda** y regresar posteriormente al chat anterior.
8. **VentasIA** permite simular una atencion comercial automatizada. Aunque es una IA basada en reglas, es suficiente para responder preguntas de catalogo, precio, stock, descuentos, medios de pago, envio y garantia.
9. El sistema queda mejor organizado porque separa el chat principal, el cliente de tienda y el servidor de tienda. Esta separacion facilita ampliar el proyecto con otros servicios en el futuro.

Anexo A: Resumen de ejecucion recomendado

1. Conectar todos los equipos a la misma red Wi-Fi o LAN.
2. Permitir en el firewall los puertos 8189 y 8190.
3. Levantar primero **Server50Tienda**, normalmente en el puerto 8190.
4. Anotar la IP real de **Server50Tienda**. Debe ser una IP privada visible, por ejemplo 192.168.0.x.
5. Levantar luego **Server50**, normalmente en el puerto 8189.
6. En **Server50**, colocar la IP y puerto de **Server50Tienda** en los campos de tienda virtual.
7. Conectar el primer cliente al chat usando IP y puerto de **Server50**.

8. Enviar mensajes y archivos para validar el chat principal.
9. Generar QR normal si se desea vincular otro dispositivo.
10. Generar QR con clonacion si se desea copiar la conexion hacia otro equipo sin anuncio normal.
11. Desde el cliente, presionar **Tienda Virtual**. La aplicacion debe cerrar la comunicacion con el chat y conectarse a **Server50Tienda**.
12. En la tienda, consultar por **catalogo, precio, stock, descuento, pago, envio o garantia**.
13. Presionar **Volver a chat anterior** para cerrar la tienda y reconectar al chat principal.

Anexo B: Puertos usados

Cuadro 8: Puertos base usados por el proyecto

Servicio	Puerto	Descripcion
Server50	8189	Chat principal. Recibe clientes Android o desktop y reenvia mensajes.
Server50Tienda	8190	Servidor de Tienda Virtual. Atiende consultas y responde mediante VentasIA .